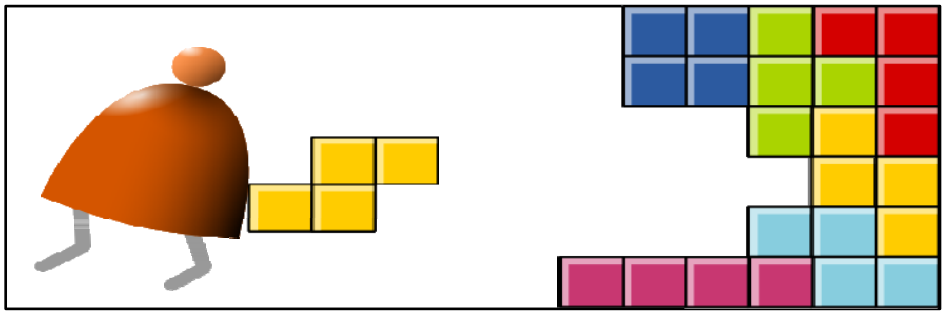
Sequential Logic



Building a Modern Computer From First Principles

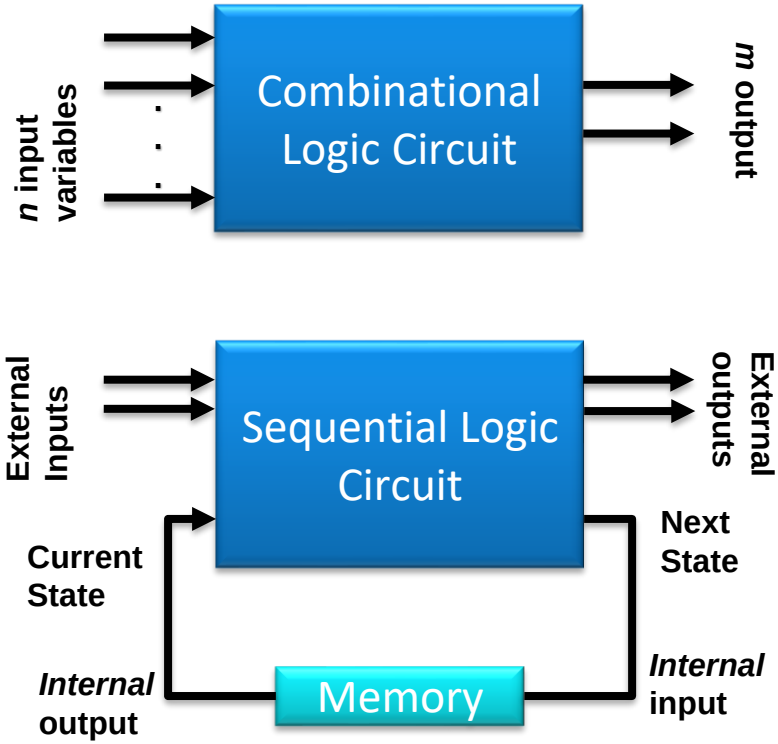
www.nand2tetris.org

Combinational vs. Sequential Logic

- **Combinational Logic** is a circuit where the output depends **only** on the current input. It has **no memory**, i.e. ALU, AND, NAND.



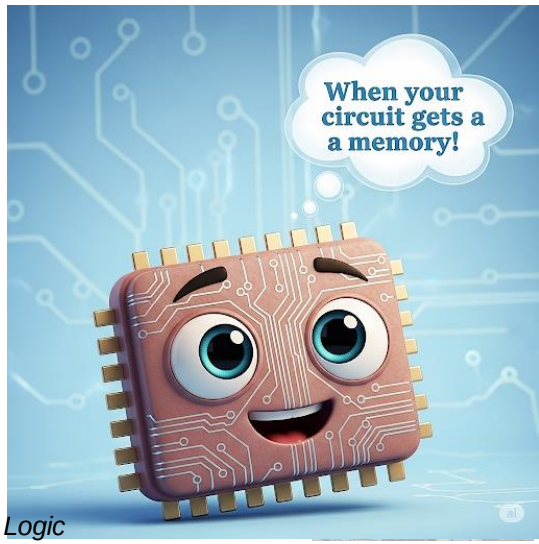
Does your calculator remember what you did yesterday?



- **Analogy:** A simple **calculator**. 2 + 2 always equals 4, no matter what you did before. It doesn't remember your last calculation after you clear the display.

- **Sequential Logic** is a circuit where the output depends on the current input **and** its **previous state** (its memory).

- **Analogy:** A **digital watch**. The time it shows is based on the current input (the clock signal) and the state it remembers from the previous second. It stores information and uses it for the next step.



Cal. Image: https://www.alamy.com/

WOOSONG UNIVERSITY
Kazakhstan

Combinational vs. Sequential Logic

- **Combinational devices:** operate on data only; provide calculation services (e.g. Nand … ALU)
- **Sequential devices:** **operate on data and a clock signal**; as such, can be made to be *state*-aware and provide storage and synchronization services
- Sequential devices are sometimes called "***clocked devices***"
- The low-level behavior of clocked / sequential gates is tricky
- The good news:
  - All sequential chips can be based on one low-level sequential gate, called "data flip flop", or DFF
  - The clock-dependency details can be encapsulated at the low-level DFF level
  - Higher-level sequential chips can be built on top of DFF gates using combinational logic only.



WOOSONG UNIVERSITY
Kazakhstan

Sequential Devices: The Role of the Clock

- **Sequential devices** operate on two inputs: **data** and a **clock signal**.
- The **clock signal** is a timing pulse that tells the device *when* to process the data.
- This makes them "**state-aware**," meaning they can remember their previous output.
- Because they can remember, sequential devices are used for **storage and synchronization**.
- For this reason, they are often called "**clocked devices**".



WOOSONG UNIVERSITY
Kazakhstan

Lecture plan

- Clock
- A hierarchy of memory chips:
  - Flip-flop gates
  - Binary cells
  - Registers
  - RAM
- Counters
- Perspective.



The Clock

- The clock in a computer is like a timer or **heartbeat** that keeps everything moving in order.
- Without it, the computer's parts would act randomly and lose coordination.

Example: Traffic Lights

- Think of a busy intersection with no traffic lights.

  Everyone moves whenever they want — chaos!

- Now add a traffic light:
  - When the light turns green → cars go.
  - When red → they stop.
  - Everyone moves in a rhythm.

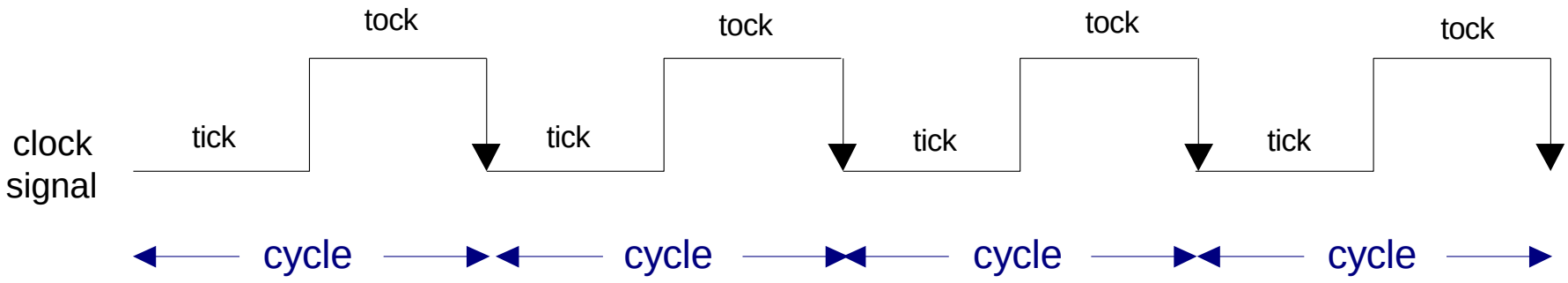


The clock in a computer does the same thing
it tells every circuit when to move (update data) and when to wait (hold data).This keeps all operations perfectly timed.



The Clock

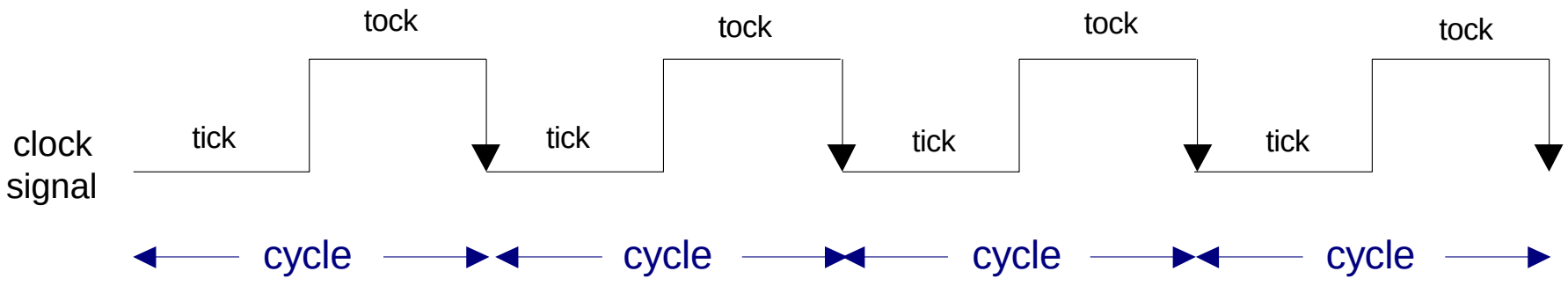
HW simulator demo



- In our jargon, a clock cycle = *tick*-phase (low), followed by a *tock*-phase (high)
- In real hardware, the clock is implemented by an **oscillator**.
- In our hardware simulator, clock cycles can be simulated either
  - Manually, by the user, or
  - "Automatically," by a test script.



The Clock

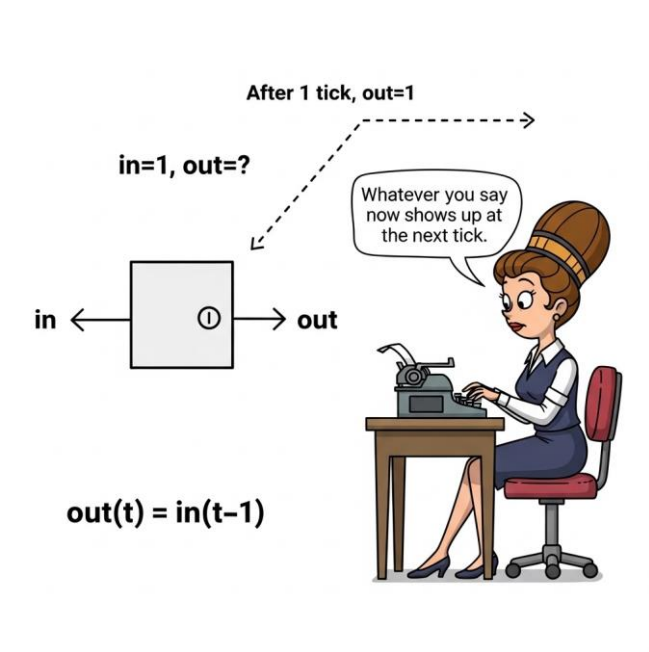


- Every CPU and memory chip follows the same clock signal.
- Each "**tick–tock**" = one cycle of work (fetch, process, store).
- Example: 3 GHz CPU → 3 billion clock beats per second.
  - Each tick tells the CPU:
    - Now fetch the next instruction.
    - Now process it.
    - Now store the result.
  - This regular rhythm keeps all the parts (CPU, memory, buses) in sync.



Flip Flop

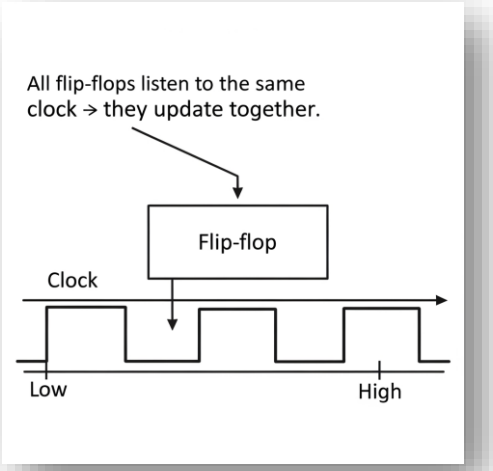
1. Show in = 1, out = ? with a label "Clock tick not happened yet."
2. After 1 tick: out = 1 because input *was* 1.
3. Change input to 0 → after next tick → out = 0.
4. Emphasize with text: **out(t) = in(t-1)**



Analogy:

- A secretary writes what you say… one minute later.
- **Text:** *Whatever you say now shows up at the next tick.*

Clock-Controlled Update





Flip Flop

- A **Flip-Flop** is a basic digital memory circuit that can store one bit of information — either 0 or 1.
- It changes its output only when the clock signal ticks, making it a synchronized memory element.



out(t) = in(t-1)

Key Points

- Acts as a one-bit storage device (remembers the last state).
- Has memory — output depends on the previous input.
- Updates its output only when triggered by the clock.
- It's the building block of registers, counters, and RAM.

Rule: out(t) = in(t−1)

- The output now equals what the input was one clock tick ago.



Flip Flop

- A Flip-Flop works like your **camera**; it captures the input only when the clock ticks.
  1. The live view keeps changing → Input (data coming in)
  2. You tap the camera button → Clock tick (store signal)
  3. The saved photo shows the last captured view → Output (stored data)



- **Rule: out(t) = in(t−1)**

  The Flip-Flop always shows what the input was one clock tick ago.



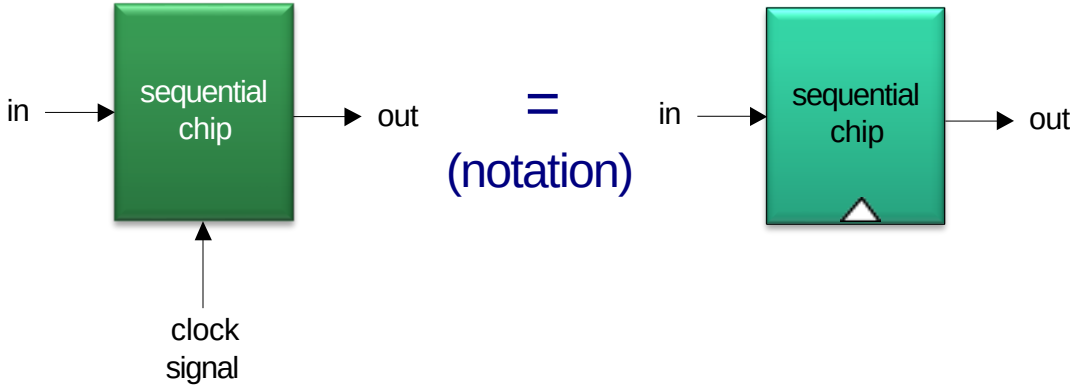
Flip-flop

HW simulator demo



out(t) = in(t-1)

- A fundamental state-keeping device
- For now, let us not worry about the DFF *implementation*
- Memory devices are made from numerous flip-flops, all regulated by the same master clock signal
- Notational convention:



WOOSONG UNIVERSITY
Kazakhstan

Flip Flop

Clock Cycle	Input (in)	Output (out)	Explanation
t=0	1	0	Before first tick, output is old value
t=1	1	1	Shows input from previous cycle
t=2	0	1	Shows input from t=1
t=3	0	0	Shows input from t=2

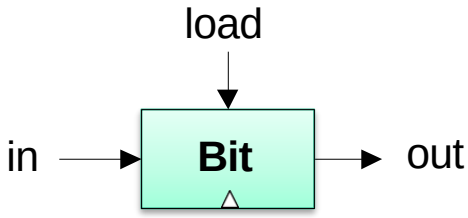


1-bit register (we call it “Bit”)

Objective: build a storage unit that can:

(a) Can be changed when you want (load = 1), and

(b) Stays the same until you decide to change it again (load = 0).



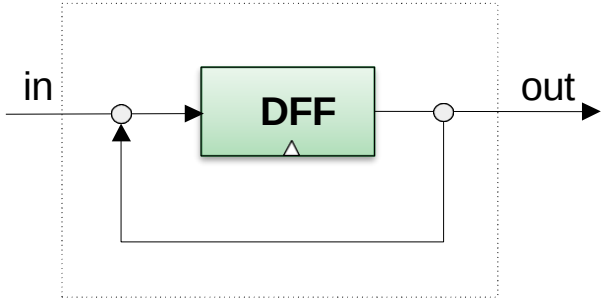
if load(t-1) then out(t)=in(t-1)
else out(t)=out(t-1)

A single DFF cannot control when to update → we need extra logic



out(t) = in(t-1)

Basic building block



out(t) = out(t-1) ?
out(t) = in(t-1) ?

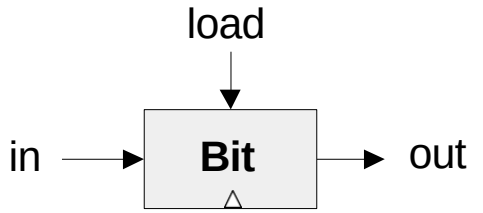
Won’t work



Bit register (cont.)

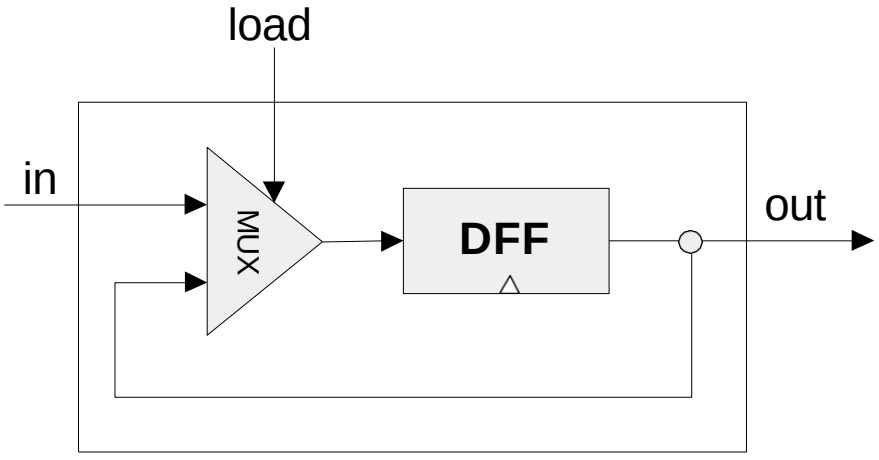
HW simulator demo

Interface



if load(t-1) then out(t)=in(t-1)
else out(t)=out(t-1)

Implementation

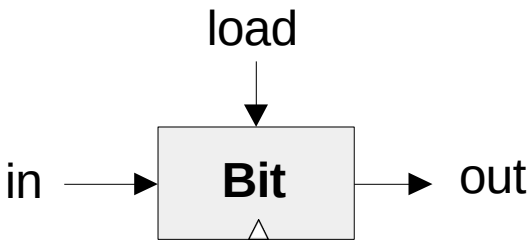


- Load bit
- Read logic
- Write logic



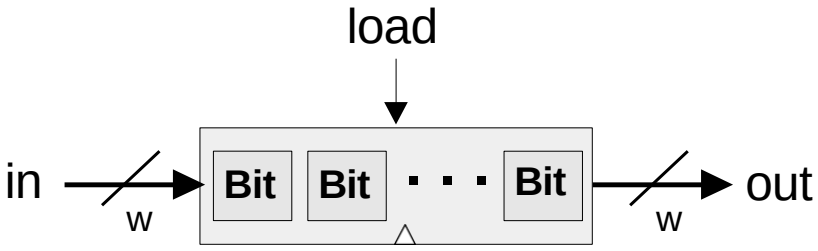
Multi-bit register

HW simulator demo



if load(t-1) then out(t)=in(t-1)
else out(t)=out(t-1)

1-bit register



if load(t-1) then out(t)=in(t-1)
else out(t)=out(t-1)

w-bit register

- Register's width: a trivial parameter
- Read logic
- Write logic



WOOSONG UNIVERSITY
Kazakhstan

Aside: Hardware Simulation

HW simulator demo

Relevant topics from the HW simulator tutorial:

- Clocked chips: When a clocked chip is loaded into the simulator, the clock icon is enabled, allowing clock control

- Built-in chips:
  - feature a standard HDL interface yet a Java implementation
  - Provide behavioral simulation services
  - May feature GUI effects (at the simulator level only).



WOOSONG UNIVERSITY
Kazakhstan

Memory

- **Memory:**
  - Main Memory: RAM... (other types, if applicable)
  - Secondary Memory: Disks... (other types, if applicable)
  - Volatile / Non-volatile
- **RAM**
  - Stores both Data and Instructions
- **Perspective**
  - Physical
  - Logical



Random Access Memory (RAM)

HW Simulator demo

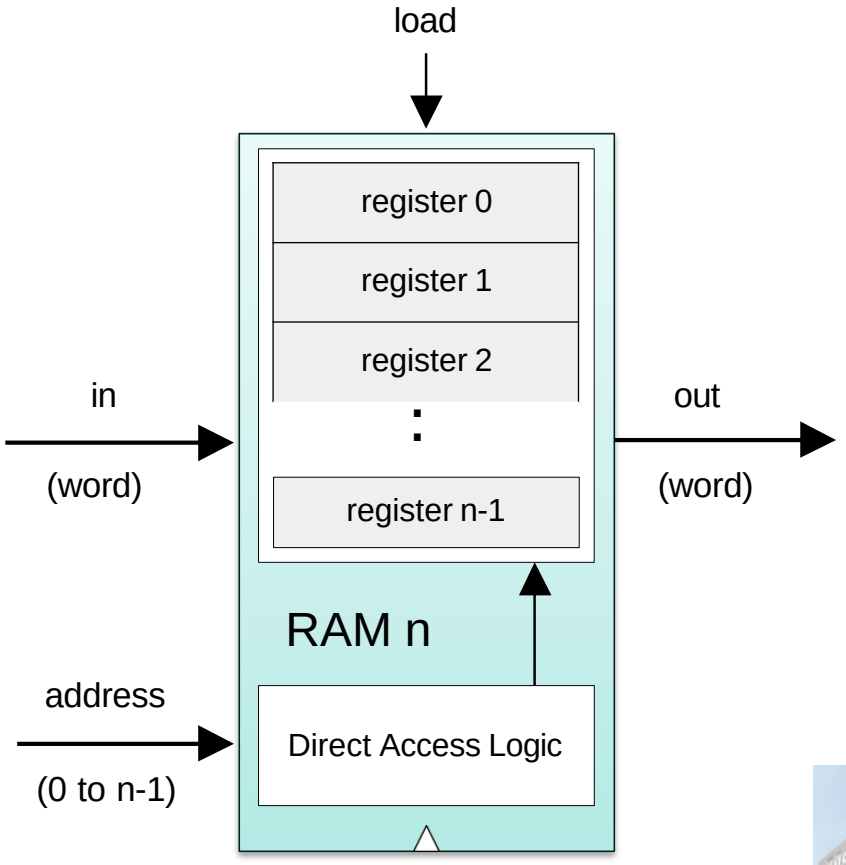
RAM abstraction: A sequence of n addressable registers. With addresses 0 to $n-1$

At any given point of time, only one register in the RAM is selected

k (width of address input): $k = \log_2 n$

w(word width): No impact on the RAM logic(Hack computer: $w=16$)

RAM is a sequential chip with a clocked behavior



- Read logic
- Write logic.



WOOSONG UNIVERSITY
Kazakhstan

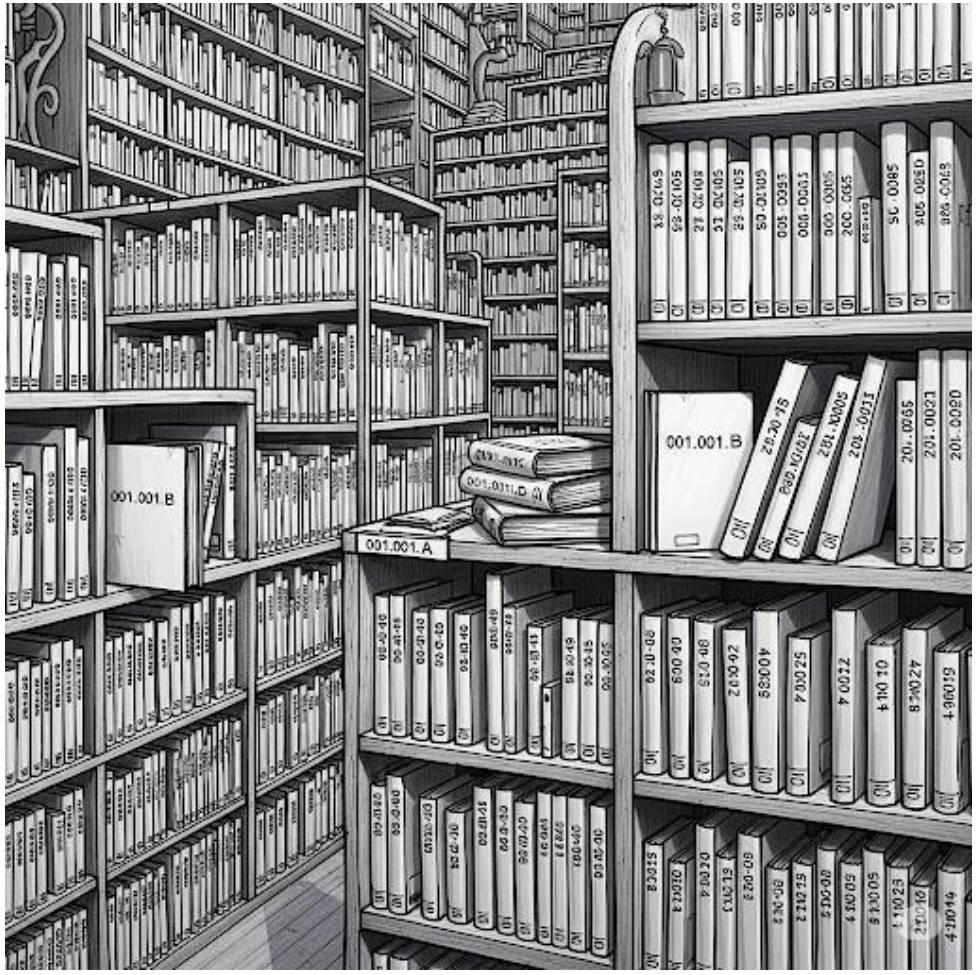
Random Access Memory (RAM)





WOOSONG UNIVERSITY
Kazakhstan

Random Access Memory (RAM)





RAM interface

RAM is made of **registers** (like RAM[i]).

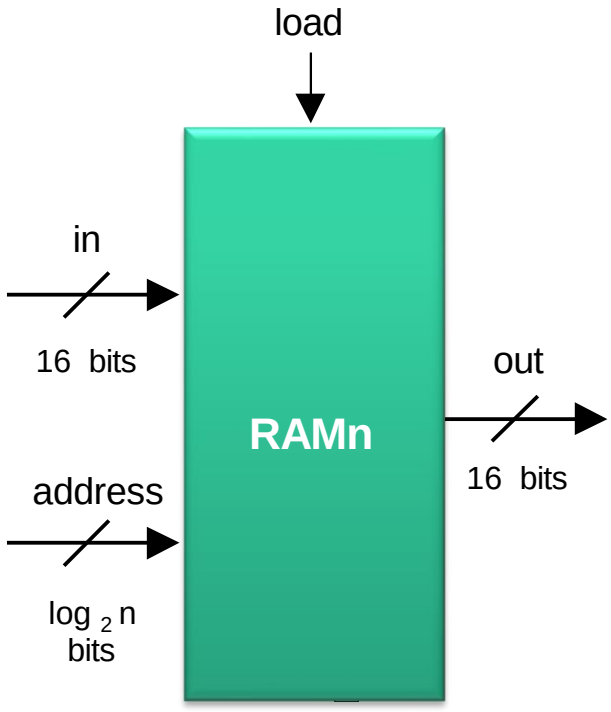
- **Write**: set the **address** = *i*, **in** = *value*, and set **load** = **1** for one clock cycle.
- **Read**: set **address** = *i* and just look at **out** (load = 0).

You never "set n" — n is just the total size of RAM. You set i (the specific address).

```
Chip name:  RAMn  // n and k are listed below
Inputs:     in[16], address[k], load
Outputs:    out[16]
Function:   out(t)=RAM[address(t)](t)
            If load(t-1) then
               RAM[address(t-1)](t)=in(t-1)
Comment:    "=" is a 16-bit operation.
```

The specific RAM chips needed for the Hack platform are:

Chip name	n	K
RAM8	8	3
RAM64	64	6
RAM512	512	9
RAM4K	4096	12
RAM16K	16384	14

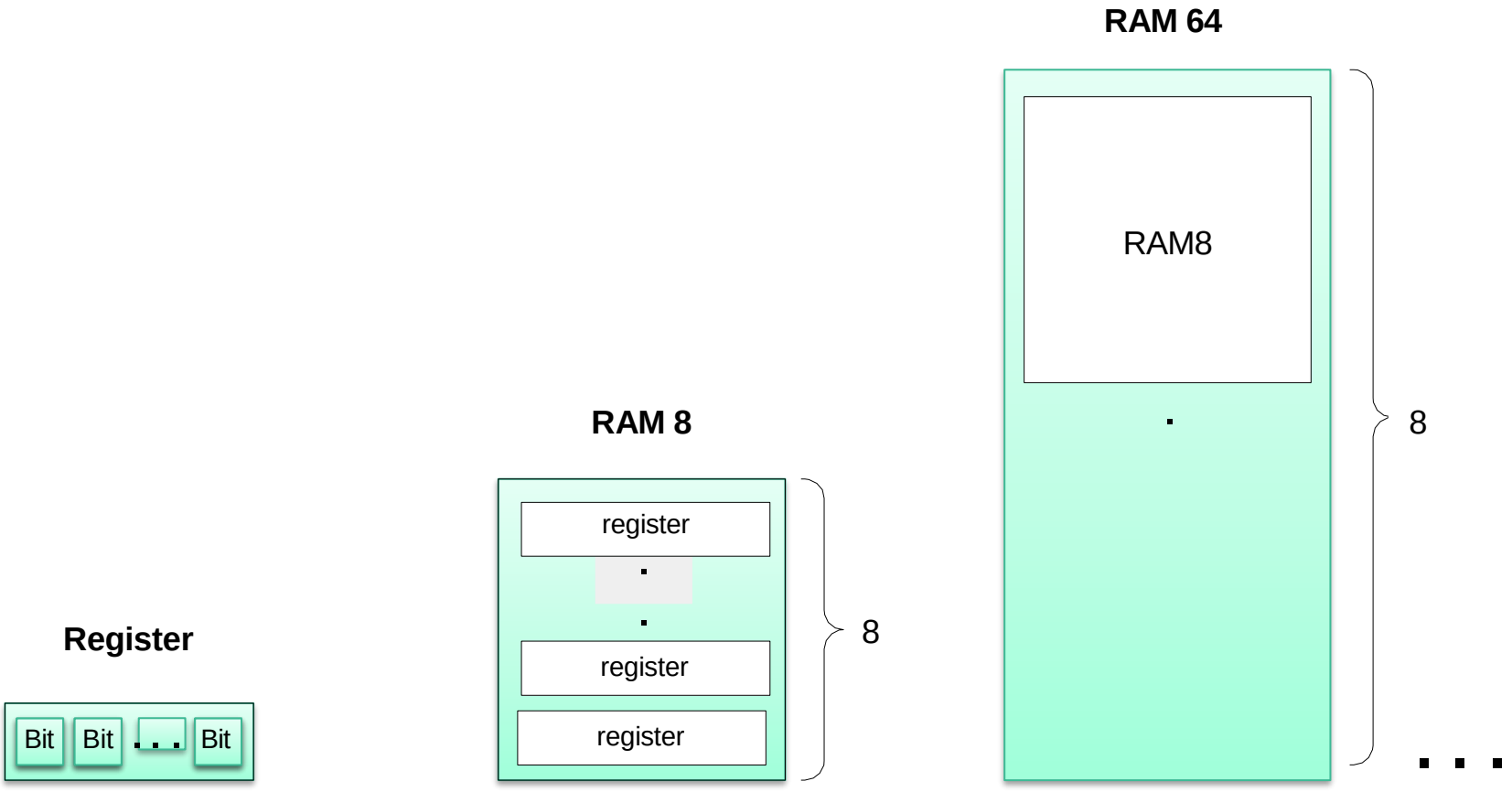


To address 8 registers, we need 3 bits (because $\log_2 8 = 3$).

To address 64 registers, you need 6 bits (because $\log_2 64 = 6$).

That's why RAM8 uses 3-bit address, and RAM64 uses 6-bit address.

RAM anatomy


RAM 64
RAM8
8
RAM 8
register
register
register
8
Register
Bit
Bit
Bit
Recursive ascent

Counter

Needed: a storage device that can:

(a) set its state to some base value

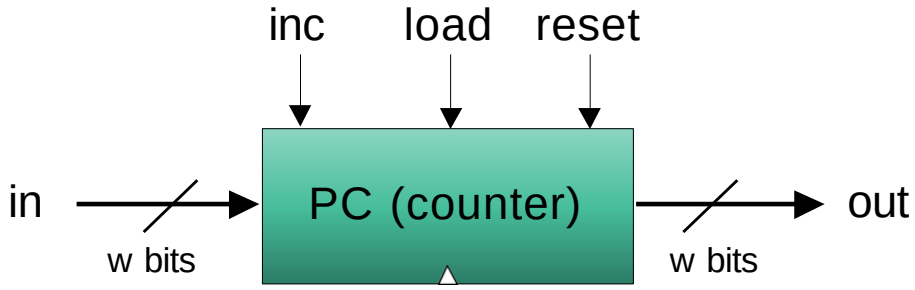
(b) increment the state in every clock cycle

(c) maintain its state (stop incrementing) over clock cycles

(d) reset its state

```
* A 16-bit counter with load and reset control bits.
* if (reset[t] == 1) out[t+1] = 0          //resetting: counter = 0
* else if (load[t] == 1) out[t+1] = in[t]  //setting: counter = value
* else if (inc[t] == 1) out[t+1] = out[t] + 1 (integer addition)  // incrementing: counter++
* else out[t+1] = out[t]                   // counter does not change
*/

CHIP PC {
IN in[16], load, inc, reset;
OUT out[16];
```



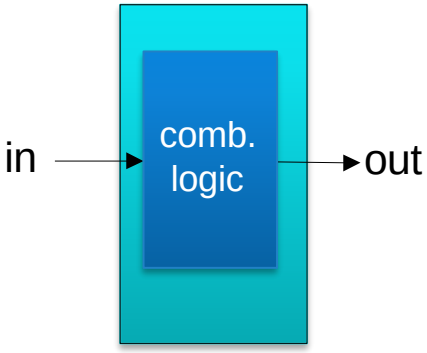
If reset(t-1) then out(t)=0
else if load(t-1) then out(t)=in(t-1)
else if inc(t-1) then out(t)=out(t-1)+1
else out(t)=out(t-1)

- Typical function: *program counter*
- Implementation: register chip + some combinational logic.



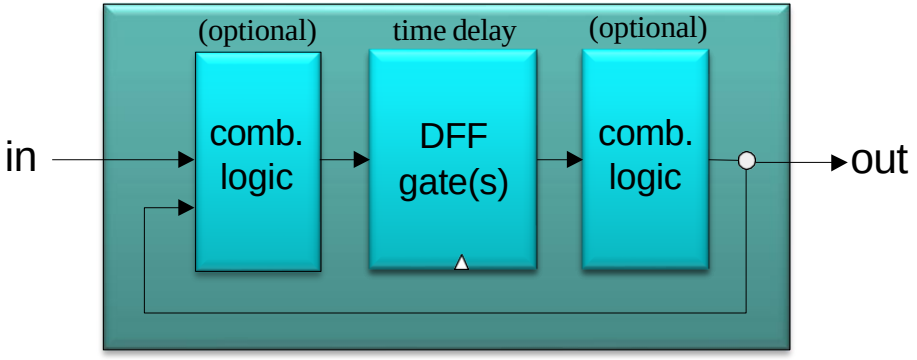
Recap: Sequential VS combinational logic

Combinational chip



out = *some function of* (in)

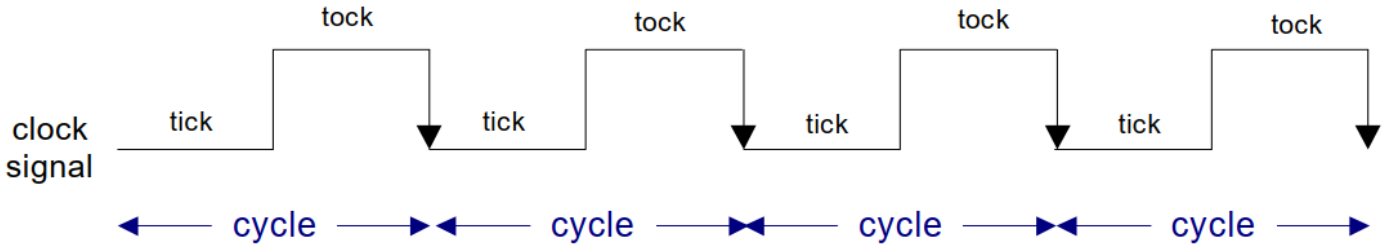
Sequential chip



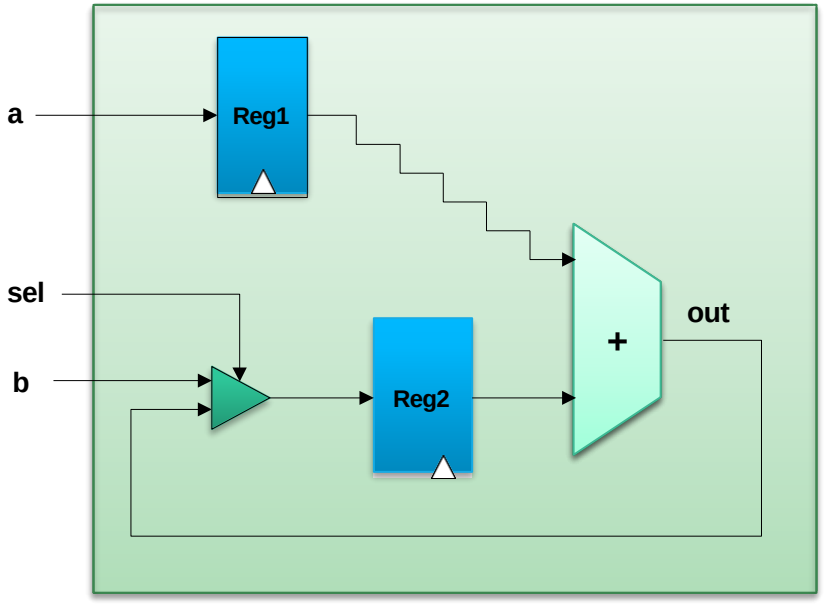
out(t) = *some function of* (in(t-1), out(t-1))



Time matters



- During a tick-tock cycle, the internal states of all the clocked chips are allowed to change, but their outputs are "latched"
- At the beginning of the next cycle, the outputs of all the clocked chips in the architecture commit to the new values.



Implications:

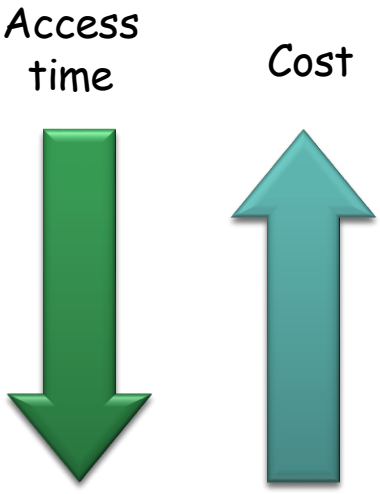
- Challenge: propagation delays
- Solution: clock synchronization
- Cycle length and processing speed.



WOOSONG UNIVERSITY
Kazakhstan

Perspective

- All the memory units described in this lecture are standard
- Typical memory hierarchy
  - SRAM ("static"), typically used for the cache
  - DRAM ("dynamic"), typically used for main memory
  - Disk



- A Flip-flop can be built from Nand gates
- But ... real memory units are highly optimized, using a great variety of storage technologies.



End notes: some poetry about the limits of logic ...

There exists a field where things are neither true nor false;
I'll meet you there.
(Rumi)

In the place where we are always right
No flowers will bloom in springtime.
(Yehuda Amichai)

A mind all logic is like a knife all blade; It makes the hand bleed that uses it.
(Rabindranath Tagor)

